

25. Създаване, обработка и трансформации на графични обекти. Допълнителни визуални ефекти.

Конструктивни операции.

1. Графични обекти - представяне и видове

Графичният обект е абстрактна структура от данни, която описва форма, положение и визуални свойства на елемент, изобразяван на екран.

Съществуват два основни модела на представяне:

- Растерно представяне - обектът е матрица от пиксели с цвeтова стойност от тях. Не носи геометрична информация, а само крайния резултат от рендериране.
- Векторно представяне - обектът се описва чрез геометрични примитиви (точки, отсечки, криви, полигони) и математически параметри (координати, радиуси, контролни точки). Позволява мащабиране без загуба на качество и последващо редактиране на геометрията.

Базови графични примитиви:

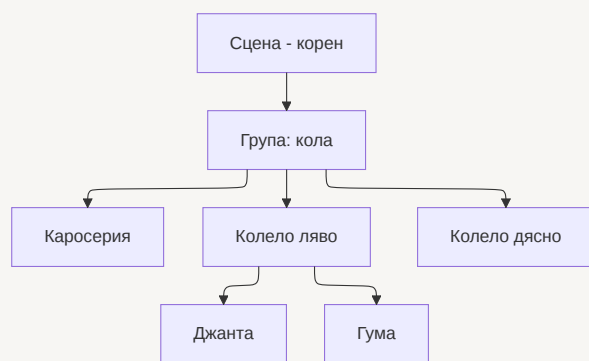
Примитив	Представяне
Точка	координати (x, y) или (x, y, z)
Отсечка	двойка крайни точки; растеризация чрез алгоритъм на Bresenham или DDA
Полигон	наредена поредица от върхове
Крива	параметрично зададена (напр. крива на Безие, B-сплайн)

Криви на Безие:

Използват се за описание на гладки, сложни криви във векторни среди като Corel Draw. Математически, една кубична крива на Безие се дефинира чрез четири контролни точки (P_0, P_1, P_2, P_3) и се описва с параметричното уравнение:

$$P(t) = (1 - t)^3 P_0 + 3(1 - t)^2 t P_1 + 3(1 - t) t^2 P_2 + t^3 P_3, \quad t \in [0, 1]$$

Сложните графични обекти обикновено се организират в сцена (scene graph) — йерархична структура, в която всеки възел носи собствена локална трансформация спрямо родителя си, а крайната позиция на обекта в света се получава чрез натрупване (композиция) на трансформациите по пътя от корена до възела.



2. Създаване и обработка на графични обекти

Създаването на обект включва задаване на геометрия (координати на примитивите) и атрибути: цвят, дебелина на контур, запълване, ниво на прозрачност. Обработката обхваща операции като:

- **Редактиране на геометрията** - добавяне/премахване на върхове, промяна на контролни точки на крива.
- **Групиране** - комбиниране на няколко обекта в единен управляем възел.

- **Подравняване и разпределение** - позициониране спрямо други обекти или спрямо канава.
- **Подредба по слоеве (z-order)** - определя реда на изчертаване и кой обект застъпва кой.

3. Геометрични трансформации

3.1 Хомогенни координати

За да се обединят линейните трансформации (ротация, мащабиране) и афинните трансформации (транслация) в единна матрична форма, в компютърната графика се въвеждат хомогенни координати. Двумерната точка (x, y) се представя като тримерен вектор $(x, y, 1)^T$, а тримерната точка (x, y, z) – като четиримерен вектор $(x, y, z, 1)^T$. Това позволява транслацията да се дефинира чрез матрично умножение вместо чрез събиране.

3.2 Базови 2D трансформации

Транслация с вектор (t_x, t_y) :

$$T = \begin{bmatrix} 1 & 0 & t_x \\ 0 & 1 & t_y \\ 0 & 0 & 1 \end{bmatrix}$$

Мащабиране с коефициенти (s_x, s_y) спрямо началото на координатната система:

$$S = \begin{bmatrix} s_x & 0 & 0 \\ 0 & s_y & 0 \\ 0 & 0 & 1 \end{bmatrix}$$

Ротация на ъгъл θ около началото:

$$R = \begin{bmatrix} \cos \theta & -\sin \theta & 0 \\ \sin \theta & \cos \theta & 0 \\ 0 & 0 & 1 \end{bmatrix}$$

Срязване (shear) по x спрямо y:

$$Sh_x = \begin{bmatrix} 1 & sh_x & 0 \\ 0 & 1 & 0 \\ 0 & 0 & 1 \end{bmatrix}$$

3.3 Композиция на трансформации

Тъй като матричното умножение е некомутиративно ($A \cdot B \neq B \cdot A$), редът на прилагане на трансформациите е от критично значение. Ако искаме да завъртим обект около произволна точка $P_c(x_c, y_c)$, а не около началото на координатната система, трябва да изпълним следната композитна трансформация:

1. Транслация на точката P_c в координатното начало: $T(-x_c, -y_c)$
2. Ротация около координатното начало: $R(\theta)$
3. Обратна транслация на обекта обратно на оригиналната му позиция: $T(x_c, y_c)$

Общата матрица на трансформация M е:

$$M = T(x_c, y_c) \cdot R(\theta) \cdot T(-x_c, -y_c)$$

Пример за композиция на трансформации в стил на графично API (OpenGL, реда на извикване съответства на реда на прилагане в обратен ред, тъй като всяко извикване умножава текущата матрица отдясно):

```
glLoadIdentity();  
glTranslatef(cx, cy, 0.0f); // 3. връщане на позицията  
glRotatef(theta, 0, 0, 1); // 2. ротация  
glTranslatef(-cx, -cy, 0.0f); // 1. преместване в началото  
drawObject();
```

3.4 3D трансформации и проекции

В пространството матриците са с размерност 4×4 и включват допълнителна ос z ; принципите на композиция са идентични. За визуализация на 3D сцена върху 2D екран се прилага **проекционна трансформация**:

- **Паралелна проекция** - запазва успоредността на линиите, използва се за технически чертежи.
- **Перспективна проекция** - обектите намаляват с отдалечаване от наблюдателя, симулира човешкото зрително възприятие; описва се с матрица, включваща деление по z компонентата (хомогенно деление).

4. Допълнителни визуални ефекти

4.1 Изглаждане на ръбове (anti-aliasing)

Растеризацията на диагонални и извити линии върху дискретна пикселна решетка води до стъпаловиден ефект (aliasing). Anti-aliasing намалява визуалния ефект чрез:

- **Суперсемплиране (supersampling)** - реднериране в по-висока резолюция и осредняване до крайния размер.
- **Покривно осредняване по площ (coverage-based)** - на всеки пиксел се присвоява стойност на прозрачност, пропорционална на частта от пикселната клетка, покрита от фигурата.

4.2 Смесване на цветове (alpha blending)

Всеки обект може да носи стойност $\alpha \in [0, 1]$, определяща прозрачност. Крайният цвят на пиксел при наслагване на обект върху фон се получава чрез линейна интерполация:

$$C_{result} = \alpha \cdot C_{src} + (1 - \alpha) \cdot C_{dst}$$

Тази операция е частен случай на по-общото семейство **Porter-Duff композиционни оператори**, определящи как се комбинират цвят и покритие на два припокриващи се слоя.

4.3 Методи за засенчване (Shading Techniques)

При 3D обекти визуалният реализъм се постига чрез модели на осветление, приложени върху нормалите на повърхността:

- Плоско сенчене (flat shading) - един цвят за цяла полигонна гран, изчислен по нейната нормала.
- Gouraud shading - цветът се изчислява във върховете и се интерполира линейно вътре в гранта.
- Phong shading - интерполират се самите нормали, а осветлението се изчислява за всеки пиксел поотделно - по-качествен, но по-скъп резултат.

4.4 Текстуриране (Texture Mapping)

Текстурирането е процес на картографиране (налагане) на 2D изображение (текстура) върху повърхността на 3D модел, за да му се придаде визуален детайл (напр. дървесна кора, тухлена стена) без увеличаване на полигоналната сложност.

- **UV Координати** - Всеки връх от 3D модела получава 2D координати (u, v) , където $u, v \in [0, 1]$. Те определят коя точно част от изображението съответства на дадената точка от геометрията.
- **Релефно текстуриране (Bump Mapping)** - Визуален ефект, който модифицира нормалните вектори на пикселите на базата на сива скала (карта на релефа). Без да се променя реалната геометрия, обектът изглежда грапав при осветяване.

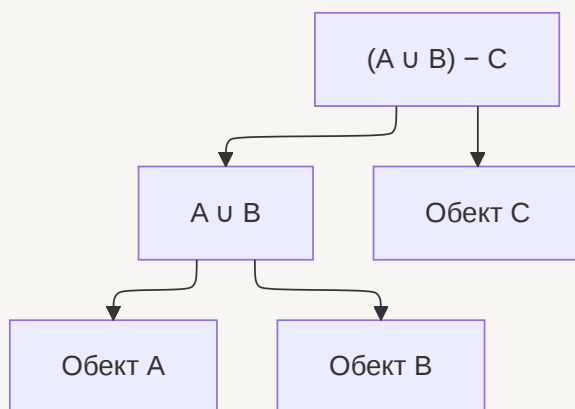
4.5 Ефекти в 2D векторна графика

В инструменти за векторно редактиране допълнителните ефекти обикновено включват: линеен/радиален градиент на запълване, размита сянка (drop shadow), контурен ефект (envelope/perspective distortion), прозрачност на слой. Тези ефекти технически са реализация на описаните по-горе принципи (blending, трансформации), приложени през удобен потребителски интерфейс.

5. Конструктивни операции

Конструктивните операции позволяват изграждане на сложни форми чрез комбиниране на прости примитиви посредством булева алгебра над множествата точки, които те заемат:

- **Обединение (Union, $A \cup B$)** - резултатът съдържа всички точки, принадлежащи на поне един от обектите.
- **Сечение (Intersection, $A \cap B$)** - само точките, общи за двата обекта.
- **Разлика (Difference, $A - B$)** - точките на A , които не принадлежат на B .
- **Симетрична разлика (XOR)** - точките, принадлежащи точно на един от двата обекта.



Резултатната форма може да се представи като **CSG дърво** - вътрешните възли са булеви операции, а листата са примитивни обекти. Обхождането на дървото от листата към корена дава крайната геометрия.

Реализацията се различава според представянето:

- **В 2D векторни редактори** (пътища/контури) операциите съответстват на *Weld* (обединение), *Trim* и *Intersect*, като алгоритмично се свеждат до полигонно изрязване (polygon clipping) - напр. алгоритъм на Weiler–Atherton, който намира пресечните точки между контурите и изгражда нов затворен контур от подходящи дъги на двата оригинала.

- В 3D графика булевите операции традиционно се изчисляват чрез **BSP-дървета (Binary Space Partitioning)** или директно върху граничното представяне (V-ger) на телата, като се намират пресечните криви между повърхностите и се прекласифицират гранните елементи спрямо това дали лежат вътре, вън или на границата на другото тяло.

Практическо предимство на CSG подхода е, че оригиналните примитиви и операциите остават запазени (нередеструктивно редактиране) — промяна на параметър на примитив автоматично предизвиква преизчисляване на крайната форма.